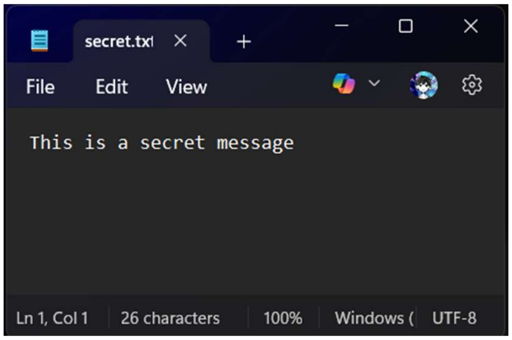
Hashing Algorithms and Digital Signatue

AES-256 Encryption

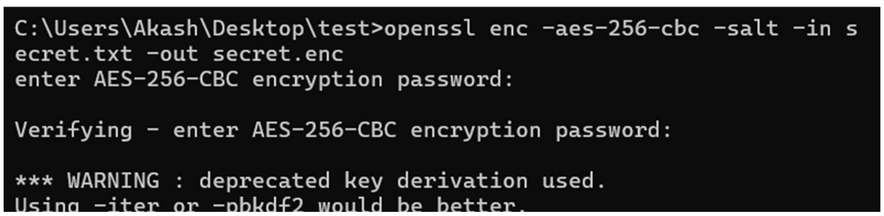
1. Creating a sample txt file.



Encrypting using the command in step 2.

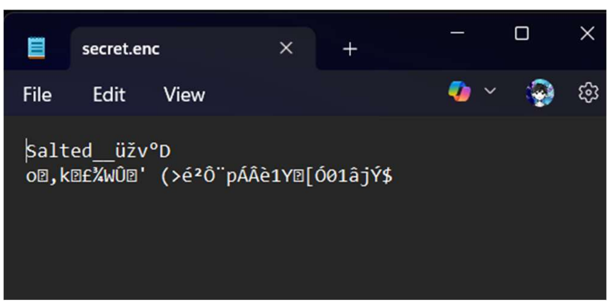
When asked for password enter the password and remember that because you"ll need this while decrypting.

2. Encrypting

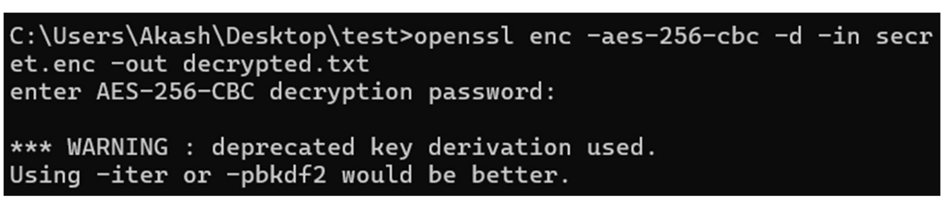


```
C:\Users\Akash\Desktop\test>openssl enc -aes-256-cbc -salt -in s
ecret.txt -out secret.enc
enter AES-256-CBC encryption password:

Verifying - enter AES-256-CBC encryption password:

*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

3. Encrypted File

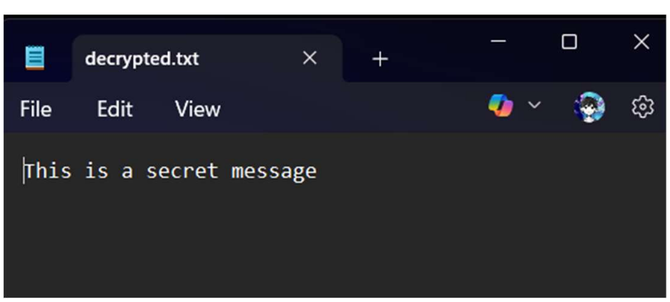


4. Decrypting



```
C:\Users\Akash\Desktop\test>openssl enc -aes-256-cbc -d -in secr
et.enc -out decrypted.txt
enter AES-256-CBC decryption password:

*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

5. Decrypted File



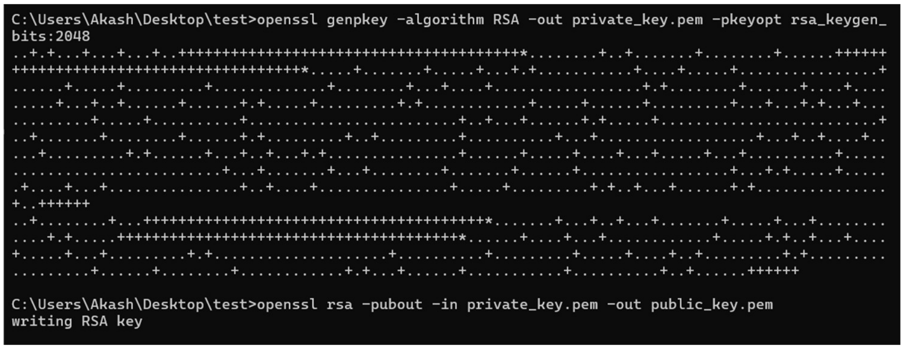
RSA Key Pair Generation

For RSA we need 2 keys

One Private Key (Decrypted using this key)

One Public Key (Encrypted using this key)

1. Generating a Private key and Generating a public key using that Private key

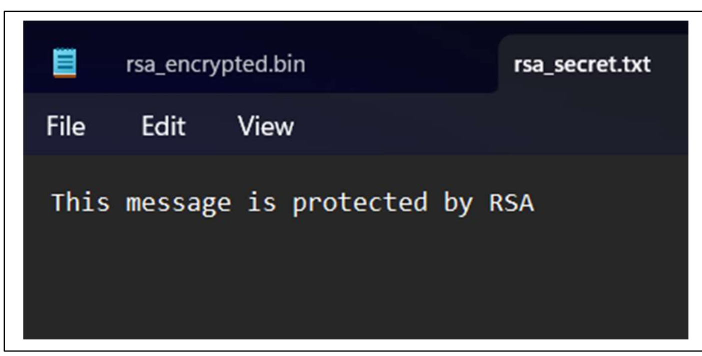


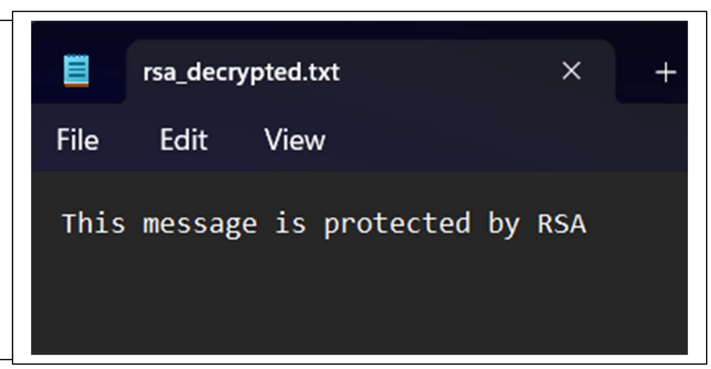
2. Encrypting using public key

```
C:\Users\Akash\Desktop\test>openssl pkeyutl -encrypt -pubin -
inkey public_key.pem -in rsa_secret.txt -out rsa_encrypted.bi
n
```

3. Decrypting using private key

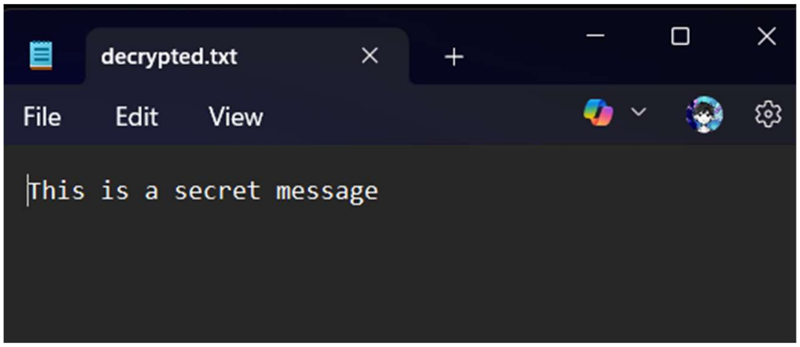
```
C:\Users\Akash\Desktop\test>openssl pkeyutl -decrypt -inkey p
rivate_key.pem -in rsa_encrypted.bin -out rsa_decrypted.txt
```



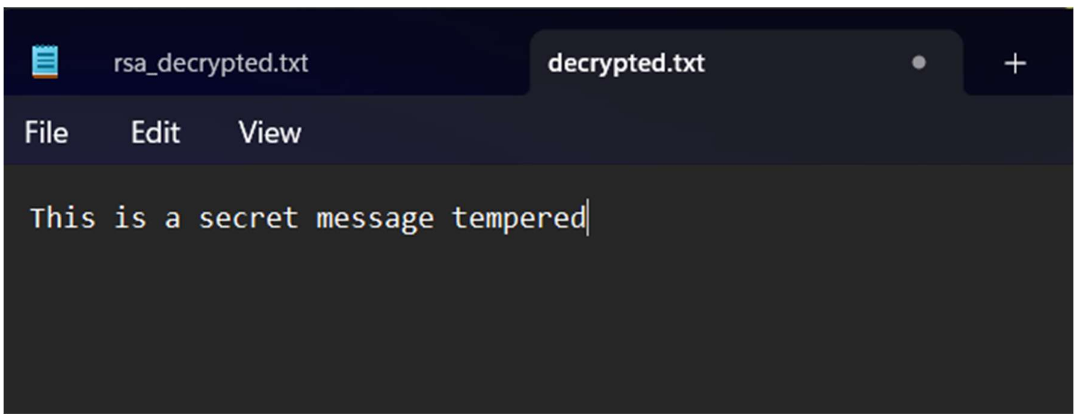


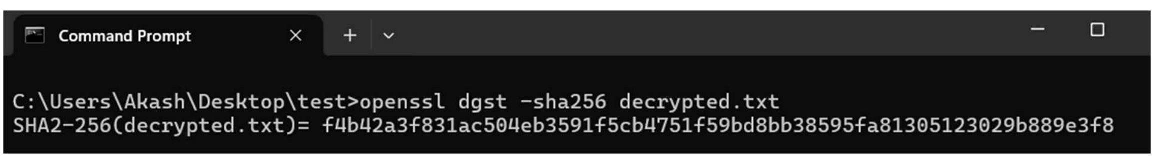
Hash a File using SHA-256

Generating a Hash of a txt file



Again Genrating the Hash for same file but after tempering it.





In this way we can check if our file has been tampered or not